



人民邮电出版社
POSTS & TELECOM PRESS

第十章 树和树的集成

机器学习与 python 实践



树和树的集成

- 树和树的集成是应用最为广泛的机器学习方法之一
深度学习之前最好的学习方法
- 集成方法相对单一的树有很大改进
Bagging、随机森林、GBDT、XGBoost

目录

- 1 分类树和回归树
- 2 Bagging 和随机森林
- 3 Boosting Trees
- 4 案例分析

目录

1 分类树和回归树

2 Bagging 和随机森林

3 Boosting Trees

4 案例分析

回归树

- 回归树本质上是一种对 f 的局部常数估计的非参数方法：

$$f(x) = \sum_{m=1}^M c_m I(x \in R_m)$$

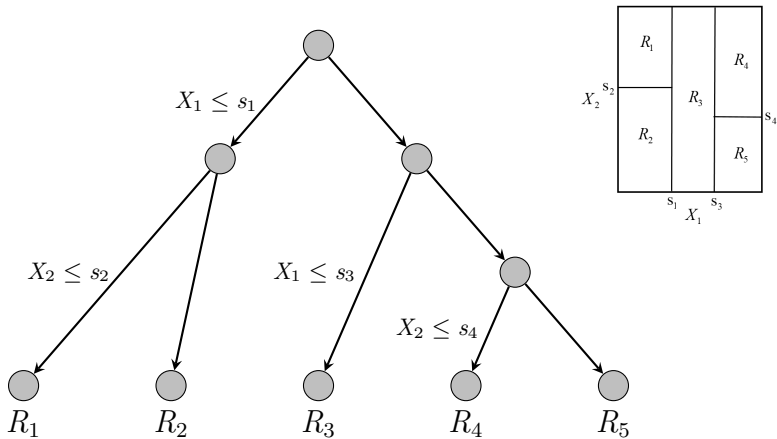
- 在平方损失 $\|Y - f(X)\|^2$ 最小准则下：

$$\hat{c}_m = \frac{1}{N_m} \sum_{x_i \in R_m} y_i$$

- 与样条方法的联系与区别

- ▶ 联系：形式上是最简单的 0 阶常数样条。
- ▶ 区别：样条方法等间隔或者按观测排序划分空间；分类树和回归树使用递归式的二分法划分空间。

回归树



回归树的生成和剪枝

• 回归树的生成

- 1 考虑切分变量 X_j 和切分点 s , 将输入空间切分为以下两个区间:

$$R_1(j, s) = \{X | X_j \leq s\}, \quad R_2(j, s) = \{X | X_j > s\}$$

变量 j 和切分点 s 可以由如下准则选取:

$$\min_{j, s} \left[\min_{c_1} \sum_{x_i \in R_1(j, s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j, s)} (y_i - c_2)^2 \right]$$

- 2 对两个子空间使用同样的方法进行切分, 递归进行, 直到达成某个条件, 如切分空间总数 (叶子节点) 达到某个上限值, 或者每个叶子节点包含的观测值小于某个下限值。切分停止时得到的树记为 T_0 。

回归树的生成和剪枝

● 回归树的剪枝 (Weakest link pruning)

- 1 移除 T_0 的一个非叶子节点 (内部节点), 移除后该节点将变成叶子节点, 得到子树 T_1 。内部节点的选择方法基于**最小化以下准则**

$$\frac{RSS(T_1) - RSS(T_0)}{|T_0| - |T_1|}$$

- 2 以同样方法对 T_1 以及后续子树进行剪枝, 直到剩下根节点的树, 记为 T_r 。由此得到树序列 $T_0 \supset T_1 \supset \cdots \supset T_r$ 。
- Weakest link pruning 得到的树序列是 Cost complexity 剪枝的解。在 Cost complexity 剪枝中, 最小化

$$\ell_\lambda(T) = \sum_{m=1}^{|T|} \sum_{\mathbf{x}_i \in R_m} (y_i - \hat{c}_m)^2 + \lambda |T|$$

分类树

- 划分空间 R_m 的类别预测

$$\hat{k}_m = \arg \max_k \hat{p}_{mk}$$

其中 $\hat{p}_{mk} = \sum_{x_i \in R_m} I(y_i = k) / N_m$ 。即观测样本中类别占比最大的类

- 常用的分类损失函数

- ▶ 错误分类率: $1 - \hat{p}_{m\hat{k}_m}$

- ▶ 基尼系数: $\sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$

- ▶ 交叉熵: $-\sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$

- 基尼指数和交叉熵通常用于分类树的生成
错误分类率通常用于分类树的剪枝

分类树的生成和剪枝

- 分类树的生成

1 考虑切分变量 j 和切分点 s , 将输入空间切分为以下两个区间

$$R_1(j, s) = X | X_j \leq s, \quad R_2(j, s) = X | X_j > s$$

变量 j 和切分点 s 可以由如下准则选取

$$\min_{j, s} \left[\frac{N_1}{N} \sum_{k=1}^K \hat{p}_{1k}(1 - \hat{p}_{1k}) + \frac{N_2}{N} \sum_{k=1}^K \hat{p}_{2k}(1 - \hat{p}_{2k}) \right]$$

2 使用递归完成分类树的生成, 直到停止条件成立

分类树的生成和剪枝

● 分类树的剪枝

- 1 移除 T_0 的一个非叶子节点 (内部节点), 移除后该内部节点将变成一个叶子节点 t^* , 得到子树 T_1 。基于**最小化以下准则**选择内部节点

$$\frac{\ell(T_1) - \ell(T_0)}{|T_0| - |T_1|} \quad or \quad \frac{\ell(t_1^*) - \ell(T^*)}{|T_1^*| - 1}$$

- 2 对子树 T_1 及后续子树使用同样过程直到 T_0 根节点, 得到树序列 $T_0 \supset T_1 \supset \dots \supset T_r$ 。

- 对应的**Cost complexity 目标函数**

$$\ell_\lambda(T) = \sum_{m=1}^{|T|} N_m(1 - \max_k \hat{p}_{mk}) + \lambda|T| = \ell(T) + \lambda|T|$$

- **剪枝准则的等价形式** (以 T_0 到 T_1 的剪枝为例):

$$\lambda_1 = \frac{\ell(t_1^*) - \ell(T_1^*)}{|T_1^*| - 1}$$

其中 t_1^* 是叶子节点, T_1^* 是 t_1^* 为根节点的树 (即 T_0 被移除的部分, 但包括 t_1^*)

分类树和回归树的剪枝和选择算法

- 1 初始化生成 T_0 , 设 $\lambda_0 = 0$
- 2 迭代 $k = 1, 2, \dots$, 直到子树仅包含一个根节点
 - 1 从 T_{k-1} 树中选择内部节点 t_k^* , 使得 $\lambda(t_k^*) \equiv \frac{\ell(t_k^*) - \ell(T^*)}{|T_k^*| - 1}$ 最小, 其中 T_k^* 是以 t_k^* 为根节点的树。
 - 2 对 T_{k-1} 剪枝, 移除 T_k^* 中除 t_k^* 之外节点, 得到 T_k 。
 - 3 设 $\lambda_k = \lambda(t_k^*)$
- 3 获得剪枝树序列 $T_0 \supset T_1 \supset \dots \supset T_r$, 以及对应的 $\lambda_0 \leq \lambda_1 \leq \dots \leq \lambda_r$
- 4 使用交叉验证法来选择子树和 λ

信息增益

- 决策树中的信息增益是由切分前类别经验分布和切分后条件分布定义的互信息。

$$\text{Information Gain} \equiv I(\mathbf{x}, \mathbf{y}) = H[p(\mathbf{y})] - CH[p(\mathbf{y}|\mathbf{x})]$$

- 假设输入空间样本量为 N , 切分为空间 R_1 和 R_2 , 样本量为 N_1, N_2 。

切分前类别的经验分布: $\hat{p} = (\hat{p}_1, \dots, \hat{p}_K)$ 。

切分后类别的经验分布: $\hat{p}_1 = (\hat{p}_{11}, \dots, \hat{p}_{1K})$ 和 $\hat{p}_2 = (\hat{p}_{21}, \dots, \hat{p}_{2K})$ 。

则

$$H[p(\mathbf{y})] = - \sum_{k=1}^K \hat{p}_k \log \hat{p}_k$$

$$CH[p(\mathbf{y}|\mathbf{x})] = E_{\mathbf{x}} H[p(\mathbf{y}|\mathbf{x})]$$

$$= -N_1/N \sum_{k=1}^K \hat{p}_{1k} \log \hat{p}_{1k} - N_2/N \sum_{k=1}^K \hat{p}_{2k} \log \hat{p}_{2k}$$

目录

- 1 分类树和回归树
- 2 Bagging 和随机森林
- 3 Boosting Trees
- 4 案例分析

Bagging

- Bagging 是集成学习和模型平均方法，用于改进机器学习方法的稳健性和精度，降低模型预测的方差。

- * 回归树：取 B 个 bootstrap 回归树在输入点 x 处预测值的平均值；
- * 分类树：选取在 B 个 bootstrap 分类树中获得最多“投票”的类。

- Bagging 对方差的改进：

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

- * Bagging 和贝叶斯的关系：Bootstrap 得到的参数估计近似于无信息先验分布下的后验估计。
- * 对比较稳健的方法改进有限，如线性回归模型。
- * 对不够稳健的方法有较大改进，如决策树、神经网络这种方差较大的非参数和高度非线性方法。

随机森林

- 随机森林是对 Bagging 方法在决策树中的一个重要改进，增加了随机特征选择这一功能。
- 随机森林算法：
 - 1 初始化参数 B, n_0
 - 2 迭代 $b = 1, 2, \dots, B$
 - 1 从特征集中随机选择 m 个特征
 - 2 从训练数据集中抽取容量为 n 的 bootstrap 样本
 - 3 通过递归二分法对选取的 m 个特征生成分类树或者回归树 T_b ，直到 T_b 叶子节点包含的观测值少于 n_0
 - 3 获得全部决策树 $\{T_b, b = 1, 2, \dots, B\}$
 - 4 模型预测：
 - 1 回归问题： $\{T_b, b = 1, 2, \dots, B\}$ 在输入点 x 处预测值的平均
 - 2 分类问题： $\{T_b, b = 1, 2, \dots, B\}$ 在输入点 x 处预测值的众数

随机森林

- 袋外数据 (out of bag sample)

对于每棵决策树，每个样本没有被抽样的概率是 $[(n-1)/n]^n$ ，当 n 较大时，大约 36.8% 没有被抽样数据称为袋外数据。

- 泛化误差的估计 (以回归树和平方损失函数为例)：

$$\widehat{\text{Err}} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \frac{1}{|C_{-i}|} \sum_{b \in C_{-i}} T_b(x_i) \right)^2$$

- 变量重要性的度量

- ▶ 基于随机扰动：

$$VI(j) = \frac{1}{B} \sum_{t=1}^B |errOOB_t - errOOB_t^*|$$

- ▶ 基于决策树分裂生长过程中由该变量切分空间导致纯度的下降幅度的平均值 (以分类树和交叉熵损失为例)：

目录

- 1 分类树和回归树
- 2 Bagging 和随机森林
- 3 Boosting Trees**
- 4 案例分析

AdaBoost

- Boosting: 通过弱分类器的组合起来提升性能, 构造强分类器。
- AdaBoost.M1 算法: 第一次实现弱分类器组合提升, 性能优异。
- 对输出为 $\{-1, 1\}$ 的弱分类器 $G_k(x)$, AdaBoost.M1 算法具体如下:

1 初始化观测样本权重 $w_i = 1/N, i = 1, 2, \dots, N$

2 迭代 $k = 1, 2, \dots, K$

2.1 对训练数据使用权重 w_i 拟合分类器 $G_k(x)$

2.2 计算

$$\text{err}_k = \frac{\sum_{i=1}^N w_i I(y_i \neq G_k(x_i))}{\sum_{i=1}^N w_i},$$

2.3 计算 $\alpha_k = \log((1 - \text{err}_k)/\text{err}_k)$,

2.4 更新权重 $w_i \leftarrow w_i \cdot \exp[\alpha_k \cdot I(y_i \neq G_k(x_i))], i = 1, 2, \dots, N$,

3 输出

$$\hat{G}(x) = \text{sign} \left(\sum_{k=1}^K \alpha_k G_k(x) \right)$$

AdaBoost

- AdaBoost.M1 算法的解释:

该算法拟合了一个可加模型 $f_K(x) = \sum_{k=1}^K \alpha_k G_k(x)$, 对应目标函数是可加模型在指数损失下的一类 forward stagewise 解。对 $k = 1, 2, \dots, K$, 逐步求解

$$\hat{\alpha}_k, \hat{G}_k(x) = \arg \min \sum_{i=1}^N \exp \left(-y_i \times (f_{k-1}(x_i) + \alpha_k G_k(x)) \right)$$

$\hat{\alpha}_k, \hat{G}_k(x)$ 与 AdaBoost.M1 算法每步计算的 $\alpha_k, G_k(x)$ 相同。

- 另一项重要研究最小角回归表明, Boosting 和 L1 正则化有紧密关系, 使用 forward stagewise 效果接近使用 L1 正则。由此提出梯度提升决策树 (GBDT) 算法和 XGBoost 方法等机器学习算法。

GBDT

- GBDT 是一种可加结构的树的集成。一般有可加结构的提升树 $f_K(x)$:

$$f_K(x) = \sum_{k=1}^K T_k(x, \theta_k)$$

其中, $T_k(x, \theta_k)$ 是决策树, $T_k(x, \theta_k) = \sum_{m=1}^{M_k} c_{mk} I(x \in R_{mk})$

- 使用逐步向前法获得 $T_1(x, \theta_1), T_2(x, \theta_2), \dots, T_K(x, \theta_K)$ 。记 $f_0 = 0$, 每步估计 θ_m 的目标函数

$$\ell(\theta_k) = \sum_{i=1}^N l(y_i, f_{k-1}(x_i) + T_k(x_i, \theta_k))$$

直接求解该目标函数一般较困难, 且过于贪婪/激进。做一阶泰勒展开近似可得

$$\ell(\theta_k) \approx \sum_{i=1}^N [l(y_i, f_{k-1}(x_i)) + g_{ik} T_k(x_i, \theta_k)] = \sum_{i=1}^N [g_{ik} T_k(x_i, \theta_k)] + Const$$

其中, $g_{ik} = \left. \frac{\partial l(y_i, f(x_i))}{\partial f(x_i)} \right|_{f(x)=f_{m-1}(x)}$

- 容易看出, $T_k(x_i, \theta_k)$ 接近 $-g_{ik}$ 可以使得 $\ell(\theta_k)$ 下降。

GBDT 算法

1 初始化 $f_0 = \arg \min_{\gamma} \sum_{i=1}^N \ell(y_i, f_{k-1}(x_i) + \gamma)$

2 迭代 $k = 1, 2, \dots, K$

2.1 对训练数据 (x_i) , 计算损失函数的梯度

$$g_{ik} = \left. \frac{\partial \ell(y_i, f(x_i))}{\partial f(x_i)} \right|_{f(x)=f_{m-1}(x)}$$

2.2 求解回归树

$$\sum_{i=1}^N (-g_{ik} - T_k(x_i, \theta_k))^2$$

得到 $\hat{\theta}_m = \{(\hat{R}_{mk}, \hat{c}_{mk}), m = 1, \dots, M_k\}$

2.3 保留 $\{\hat{R}_{mk}, m = 1, \dots, M_k\}$, 重新估计 c_{mk}

$$\tilde{c}_{mk} = \arg \min_{\gamma} \sum_{x_i \in \hat{R}_{mk}} \ell(y_i, f_{k-1}(x_i) + \gamma)$$

2.4 更新 $f_k(x) = f_{k-1}(x) + \sum_{m=1}^{M_k} \tilde{c}_{mk} I(x \in \hat{R}_{mk})$

3 输出 $f(x) = f_M(x)$

GBDT Tips

- 专家建议每棵树的叶子节点数量在 4-8 之间。
- GBDT 中的变量重要性也可类似随机森林进行计算。
- 改进方法
 - ▶ 引入正则化方法可以进一步提高 GBDT 的性能，例如在算法第 2.4 步更新的时候加入学习率，

$$f_k(x) = f_{k-1}(x) + \eta \sum_{m=1}^{M_k} \tilde{c}_{mk} I(x \in \hat{R}_{mk})$$

- ▶ 随机梯度提升：在训练过程中可使用类似 Bagging 使用 bootstrap 样本的思路，或随机无放回的抽取部分数据来训练 $T_k(x_i, \theta_k)$ ，实现降低每棵树之间的相关性，从而降低估计方差。
- ▶ Column Subsampling：参照随机森林在每一步中使用随机特征，进一步降低每棵树之间的相关性。

XGBoost

- XGBoost 是 Extreme Gradient Boosting 的简写，是对 GBDT 算法的一种高效改进。

1. 与 GBDT 相比，XGBoost 对目标函数展开到二阶，

$$\ell(\theta_k) \approx \sum_{i=1}^N \left[l(y_i, f_{k-1}(x_i)) + g_{ik} T_k(x_i, \theta_k) + \frac{1}{2} h_{ik} (T_k(x_i, \theta_k))^2 \right]$$

其中， $h_{ik} = \frac{\partial^2 \ell(y_i, f(x_i))}{\partial f(x_i)^2} \Big|_{f(x)=f_{m-1}(x)}$

2. 显式的引入正则化惩罚项，第 k 步的目标函数为

$$Obj(\theta_k) = \sum_{i=1}^N \left[g_{ik} T_k(x_i, \theta_k) + \frac{1}{2} h_{ik} (T_k(x_i, \theta_k))^2 \right] + \Omega(T_k)$$

3. 在树的生成方面，XGBoost 与 GBDT 类似，但增加了预剪枝的处理，防止过拟合。

XGBoost 求解

二阶泰勒展开后的极值求解

- ▶ 将 $T_k(x, \theta_k) = \sum_{m=1}^{M_k} c_{mk} I(x \in R_{mk})$ 重新定义一个等价形式

$$T_k(x, \theta_k) = c_{q(x), k}$$

其中, $q(x) = m \Leftrightarrow x \in R_{mk}$ 。

- ▶ 假设正则项 $\Omega(T_k)$ 有如下形式

$$\Omega(T_k) = \gamma |T_k| + \frac{1}{2} \lambda \sum_{m=1}^{M_k} c_{mk}^2$$

- ▶ 令 $I_{mk} = \{x_i | x_i \in R_{mk}\}$, $i \in I_{mk} \Leftrightarrow x_i \in R_{mk}$, 目标函数即

$$Obj(\theta_k) = \sum_{m=1}^{M_k} \left[G_{mk} \times c_{mk} + \frac{1}{2} (\lambda + H_{mk}) \times c_{mk}^2 \right] + \gamma |T_k|$$

其中 $G_{mk} = \sum_{i \in I_{mk}} g_{ik}$, $H_{mk} = \sum_{i \in I_{mk}} h_{ik}$

XGBoost 求解

- 二阶泰勒展开后的极值求解（续）

- ▶ 给定树的区间划分 R_{mk} , $m = 1, 2, \dots, M_k$ 下, c_{mk} 的最优解为

$$\hat{c}_{mk} = -\frac{G_{mk}}{H_{mk} + \lambda}$$

- ▶ 对应的目标函数的极小值为

$$\hat{Obj}(\theta_k) = -\sum_{m=1}^{M_k} \frac{G_{mk}^2}{H_{mk} + \lambda} + \gamma |T_k|$$

第一项度量了 T_k 结构的好坏, 第二项是对树的大小的惩罚。

- ▶ **对比:** GBDT 中 $\hat{c}_{mk}$ 的解为均值, 即 $\hat{c}_{mk} = G_{mk} = \sum_{i \in I_{mk}} g_{ik}$ 。

XGBoost 求解

● 预剪枝

- ▶ XGBoost 使用递归二分法进行树的生长，遍历所有特征和节点选取**最优的划分变量**。比较准则是**划分前后目标函数值的增益最大**
- * 考虑切分变量 X_j 和切分点 s ，将输入空间 R_{mk} 分为以下两个区间：

$$R_{mk1}(j, s) = \{X \in R_{mk} | X_j \leq s\}$$

$$R_{mk2}(j, s) = \{X \in R_{mk} | X_j > s\}$$

数据集为 $I_{mk1} = \{\mathbf{x}_i | \mathbf{x}_i \in R_{mk1}\}$, $I_{mk2} = \{\mathbf{x}_i | \mathbf{x}_i \in R_{mk2}\}$

- * 变量 j 和切分点 s 可以由最大化如下增益准则选取：

$$\frac{1}{2} \left[\frac{G_{mk1}^2}{H_{mk1} + \lambda} + \frac{G_{mk2}^2}{H_{mk2} + \lambda} - \frac{G_{mk}^2}{H_{mk} + \lambda} \right] - \gamma$$

其中, $G_{mkr} = \sum_{i \in I_{mkr}} g_{ik}$, $H_{mkr} = \sum_{i \in I_{mkr}} h_{ik}$, $r = 1, 2$

- ▶ 上式为切分后目标函数极值增益，**增益为负则停止分裂，即预剪枝。**

方法对比

- **Bagging**: Bootstrap in Trees
- **随机森林**: Bagging+ 随机特征选择 (Column Subsampling)
- **AdaBoost**: Boosting in Trees
 - ▶ 改变误样本的权重, 进行 boosting。
 - ▶ 过程可以看作一个损失函数为指数损失的 FS, 而 FS 类似于 L1。
- **GBDT**: AdaBoost 的推广形式, 损失函数形式需要一阶可导
 - ▶ 损失函数可以不止是指数损失。
 - ▶ 对损失函数进行一阶泰勒展开近似。
 - ▶ 求解过程用负梯度近似残差。
- **XGBoost**: GBDT 的改进, 损失函数形式需要二阶可导
 - ▶ 对损失函数泰勒展开到二阶;
 - ▶ 损失函数显式使用惩罚项;
 - ▶ 加入预剪枝过程。

目录

- 1 分类树和回归树
- 2 Bagging 和随机森林
- 3 Boosting Trees
- 4 案例分析**

案例分析

- **目的**：预测股票涨跌概率，并形成交易策略。
- **数据选取**：2013 年 11 月 18 日 -2015 年 12 月 9 日沪深 300 成分股的日收益率数据，即 300 只股票 505 天的收益率数据。
- **特征选取**：前 5 日收益率，即 5 维特征。
- **训练集**：
将 2013 年 11 月 18 日 -2015 年 9 月 23 日共计 455 天的沪深 300 成分股日收益数据整理形成 5 维特征；
将 2013 年 11 月 25 日 -2015 年 9 月 23 日共计 450 天的沪深 300 成分股日收益数据按涨跌整理形成 01 标签。
- **测试集**：
将 2015 年 9 月 17 日 -2015 年 12 月 9 日共计 55 天的沪深 300 成分股日收益数据整理形成 5 维特征；
将 2015 年 9 月 24 日 -2015 年 12 月 9 日共计 50 天的沪深 300 成分股日收益数据按涨跌整理形成 01 标签。
- **模型**：随机森林、GBDT、XGBoost
- **策略**：1、纯多头策略；2、多空策略。

案例分析

● 随机森林 (n_estimators=10,min_samples_leaf=2)

		precision	recall	f1-score	support
Train	0	0.96	0.97	0.97	70363
	1	0.97	0.96	0.96	64637
	avg/total	0.97	0.97	0.97	135000
Test	0	0.55	0.58	0.56	7600
	1	0.54	0.52	0.53	7400
	avg/total	0.55	0.55	0.55	15000

● GBDT (n_estimators=10, max_depth=2, min_samples_split=2, lr=0.1)

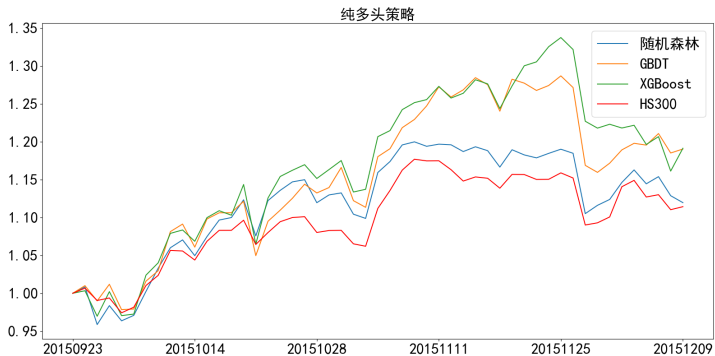
		precision	recall	f1-score	support
Train	0	0.54	0.71	0.63	70363
	1	0.55	0.39	0.45	64637
	avg/total	0.55	0.56	0.54	135000
Test	0	0.54	0.71	0.61	7600
	1	0.56	0.38	0.45	7400
	avg/total	0.55	0.55	0.53	15000

● XGBoost (lr=0.1)

		precision	recall	f1-score	support
Train	0	0.60	0.61	0.60	70363
	1	0.57	0.55	0.56	64637
	avg/total	0.58	0.58	0.58	135000
Test	0	0.56	0.55	0.56	7600
	1	0.55	0.56	0.55	7400
	avg/total	0.56	0.55	0.55	15000

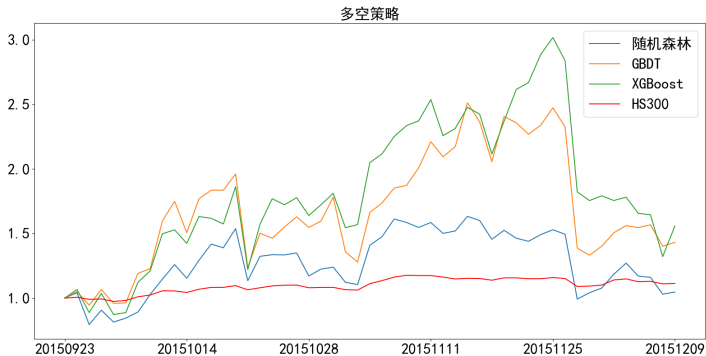
案例分析

纯多头策略：选取预测上涨概率最高的 10 只股票做多，仓位平均。



案例分析

多空策略：选取预测上涨概率最高的 10 只股票做多，选取预测下跌概率最高的 10 只股票做空，仓位平均。



结语

- 树是一种特殊的非参数估计方法（局部常数估计）
- 树的集成通过减少树之间的相关性降低估计方差
- 梯度提升树直接或者间接运用的多种有效的方法，例如 L1、L2 正则、Bootstrap、Column Sampling 等
- 算法成熟、在中小型数据中表现优异